CHEATSHEET SEMANTIC WEB · mang vào phòng thi

*Đọc câu hỏi → bảng dưới tìm mã dạng → lật tới khung **D…** → chép khuôn, thay dữ liệu của đề. Bản đầy đủ: TAI-LIEU-DAY-DU.pdf (không cần in).*

0 · BẢN ĐỒ ĐỀ THI: dấu hiệu → dạng → mục

Bức tranh chung — 5 việc: ① **Dữ liệu** RDF/Turtle (D1, D2) → ② **Định nghĩa** RDFS/OWL (D3, D5) → ③ **Suy diễn** bằng reasoner: luật RDFS, OWL, tableau, Datalog (D4, D6, D12, D7, D11) → ④ **Hỏi** SPARQL + đại số (D8, D9) → ⑤ **Kiểm tra** SHACL (D10).

Đề có câu…	Dạng	Mục
"Tạo RDF / Turtle **và vẽ đồ thị**" cho đoạn văn	Câu văn → Turtle + đồ thị	**D1** · tr.1
"Biểu diễn đồ thị sau bằng Turtle / vẽ đồ thị cho triple / RDF-XML"	Chuyển đổi	**D2** · tr.2
"Dùng RDF/RDFS biểu diễn: URI1 là lớp, có domain…"	RDFS schema	**D3** · tr.2
"Viết bộ-3 **suy ra được** (RDFS)", "đúng hay sai"	Suy diễn RDFS	**D4** · tr.2
"Định nghĩa lớp/thuộc tính bằng **RDFS + OWL**", "biểu thức **DL**"	OWL + DL	**D5** · tr.2
"Tầng nào (RDFS hay OWL)… bổ sung ràng buộc để suy ra…"	Suy diễn OWL	**D6** · tr.3
"Chứng minh bằng **tableau**", NNF, consistency	Tableau	**D12** · tr.3
"Chương trình **Datalog**… **cây chứng minh**… **điểm cố định**"	Datalog	**D7** · tr.3
"Viết truy vấn **SPARQL**" (câu 2.5đ)	SPARQL	**D8** · tr.3
"Viết **biểu thức đại số SPARQL**, giải thích từng bước"	Đại số	**D9** · tr.4
"Tạo **khuôn hình SHACL**", "nút nào được kiểm / vi phạm"	SHACL	**D10** · tr.4
"Chức năng của **reasoner** trong KB", câu lý thuyết	Lý thuyết	**D11** · tr.4

Prefix chép đầu bài:

```
@prefix rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix owl:  <http://www.w3.org/2002/07/owl#> .
@prefix xsd:  <http://www.w3.org/2001/XMLSchema#> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .
@prefix sh:   <http://www.w3.org/ns/shacl#> .
@prefix :     <http://example.org/> .
```

Khi nào dùng prefix nào:

Prefix	Dùng khi nào
`rdf:`	cú pháp lõi: `a` (= `rdf:type`), `rdf:Property`, reification (`Statement/subject/predicate/object`), collection (`first/rest/nil`)
`rdfs:`	schema cơ bản: `Class`, `subClassOf`, `subPropertyOf`, `domain`, `range`, `label`, `comment`
`owl:`	câu hỏi thuộc phần OWL/DL (D5-D6): `equivalentClass/Property`, `disjointWith`, `unionOf/intersectionOf/complementOf`, `Restriction` (`someValuesFrom/allValuesFrom/hasValue`), cardinality, `inverseOf`, `TransitiveProperty/SymmetricProperty/FunctionalProperty`, `sameAs`
`xsd:`	literal có kiểu cụ thể: `"10"^^xsd:integer`, `"2024-01-01"^^xsd:date`, `xsd:float`. Số/chuỗi đơn giản thì khỏi cần gắn.
`foaf:`	đề nói về người/quan hệ xã hội và **có cho sẵn** prefix này: `foaf:name`, `foaf:knows`, `foaf:member`. Đề không cho thì tự định nghĩa property riêng dưới `:`.
`dc:` / `dcterms:`	metadata mô tả tài liệu (sách, bài báo): `dc:title`, `dc:creator`, `dc:date`. Ít gặp — chỉ thêm khi đề nhắc "tác giả/tiêu đề/ngày xuất bản" của một tài liệu.
`sh:`	chỉ dùng ở phần SHACL (D10): `NodeShape`, `targetClass`, `property`, `minCount/maxCount`, `datatype`, `pattern`...

Đề cho sẵn prefix nào thì dùng đúng cái đó; đề không cho thì chỉ cần `rdf, rdfs, owl, xsd` là đủ cho hầu hết câu, các prefix còn lại thêm khi ngữ cảnh gợi ý.

Mẹo chung: ghi rõ IRI tự đặt · mỗi bước có lý do (tên luật / tên phép) · không chắc thì viết **cả 2 cách** và nói khác nhau ở đâu.

D1 · Câu văn → Turtle + vẽ đồ thị

Đề nói…	Dùng
"A **nghĩ / cảm thấy / nhận xét / cho rằng** X"	**reification** `[a rdf:Statement; rdf:subject…; rdf:predicate…; rdf:object…]`
"**gồm / danh sách / nhóm tác giả / xếp hạng**"	**collection** `( :a :b :c )` → rdf:first/rest/nil (thứ tự: `rdf:Seq`)
"đạt **điểm 10 môn X**", quan hệ ≥ 3 thành phần	**n-ary**: `:sv :score [ :value 10 ; :course :X ]`
"một người nào đó / không rõ tên"	blank node `[]`
tên, số, ngày	literal: `"Ann"`, `10`, `"2024-01-01"^^xsd:date`, `"Avatar"@en`

Khuôn: ① prefix → ② đặt IRI cho từng thực thể → ③ Turtle → ④ vẽ: **IRI = elip, literal = chữ nhật, blank = elip rỗng**, mỗi triple = 1 mũi tên có nhãn → ⑤ đếm triple nếu đề giới hạn.

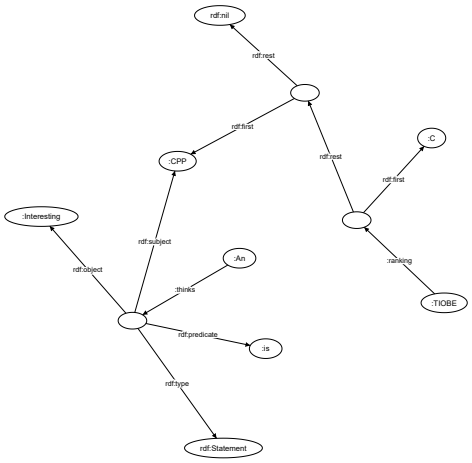
Liệt kê HẾT triple (Notes): với mỗi thực thể: `a :Lớp` · tên/nhãn (literal) · mọi quan hệ trong câu · thuộc tính số/ngày có kiểu · quan hệ ngược nếu câu hàm ý · reification nếu là ý kiến. Vị trí: **S** = IRI/blank · **P** = chỉ IRI · **O** = IRI/blank/literal.

Mẫu (2022-C1 — đáp án thầy):

```
@prefix b: <http://exam.hust.edu.vn/book/> .
b:nguyen-van-a b:name "Nguyễn Văn A" ;
  b:feels [ a rdf:Statement ; rdf:subject b:design-patterns ;
            rdf:predicate rdf:type ; rdf:object b:GoodBook ] .
b:design-patterns a b:Book ; b:title "Design Patterns: …" ;
  b:writtenBy ( b:erich-gamma b:richard-helm b:ralph-johnson
b:john-vlissides ) .
b:erich-gamma b:name "Erich Gamma" .   # … tương tự 3 tác giả
còn lại
```

Hình mẫu reification + collection:

```
@prefix : <http://example.org/> .
:An :thinks [ a rdf:Statement ; rdf:subject :CPP ;
rdf:predicate :is ; rdf:object :Interesting ] .
:TIOBE :ranking ( :C :CPP ) .
```



2023-C1 (thầy): `:sw_course :ref (:foswt :learning_sparql :sw_ontologist); rdfs:label "The Semantic Web".` · `:nva a :Student; :score [:value 10; :course :sw_course].` · `[a :Teacher; :course :sw_course] :comment [a rdf:Statement; rdf:subject :nva; rdf:predicate rdf:type; rdf:object :ExcellentStudent].`

D2 · Đồ thị ↔ Turtle ↔ N-Triples ↔ RDF/XML

- **Đồ thị → Turtle:** đi từng nút có mũi tên ra = subject; gom cạnh cùng subject bằng `;`, cùng predicate bằng `,`. Nút **không có tên / chữ thường** ("a dragon") = **blank node**.
- **Turtle → đồ thị:** bung `;` `,` `[]` `()` thành từng triple, vẽ từng cạnh. `[]` = elip rỗng; `()` = chuỗi elip rỗng rdf:first/rest → rdf:nil.
- **N-Triples:** mỗi dòng 1 triple, IRI đầy đủ <…>, blank `_:b1`, kết thúc `.`

P1 Ex4 (Georges/Michael — 9 triple):

```
:Georges a :Saint ; foaf:name "Georges" ; :defeated [ a :Dragon ] .
:Michael a :Saint , :Archangel ; foaf:name "Michael" ;
:defeated :Satan .
:Satan foaf:name "Satan" .
```

Khung RDF/XML:

```
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
xmlns:ex="http://example.org/">
  <rdf:Description rdf:about="http://example.org/Michael">
    <rdf:type rdf:resource="http://example.org/Archangel"/>
<!-- IRI object -->
    <foaf:name>Michael</foaf:name>
<!-- literal -->
    <ex:defeated rdf:resource="http://example.org/Satan"/>
    <ex:knows rdf:nodeID="b1"/>
<!-- blank node -->
  </rdf:Description>
  <rdf:Description rdf:nodeID="b1"><rdf:type
rdf:resource="http://example.org/Dragon"/></rdf:Description>
</rdf:RDF>
```

P1 Ex6 (`[] foaf:knows :Minh; foaf:member [a :Company]`): 2 blank node; `:teaches "Semantic Web", "Data Integration"` = 2 cạnh tới 2 chữ nhật.

D3 · RDFS schema

Câu	RDFS
X là lớp	`:X a rdfs:Class .`
p là thuộc tính	`:p a rdf:Property .`
a là thể hiện của X	`:a a :X .`
X là lớp con của Y	`:X rdfs:subClassOf :Y .`
p là thuộc tính con của q	`:p rdfs:subPropertyOf :q .`
p có miền X, khoảng Y	`:p rdfs:domain :X ; rdfs:range :Y .`
nhãn / chú thích	`rdfs:label "…"@vi ; rdfs:comment "…"`

P1 Ex3: `:URI1 a rdfs:Class . :URI2 a rdfs:Class . :URI3 a rdf:Property ; rdfs:domain :URI1 ; rdfs:range :URI2 . :URI4 a :URI1 . :URI5 a :URI2 . :URI6 a :URI2 .` **Giới hạn RDFS** (lý do phải sang OWL): không có phủ định/disjoint, không cardinality, không inverse/transitive, domain/range chỉ toàn cục (không ràng buộc theo từng lớp), không định nghĩa lớp bằng điều kiện.

D4 · Suy diễn RDFS

Luật	Nếu có	Suy ra
rdfs2	`p rdfs:domain C` + `x p y`	`x a C`
rdfs3	`p rdfs:range C` + `x p y`	`y a C` (y không là literal)
rdfs5	`p ⊑ q`, `q ⊑ r` (subPropertyOf)	`p subPropertyOf r`
rdfs7	`p subPropertyOf q` + `x p y`	`x q y`
rdfs9	`C subClassOf D` + `x a C`	`x a D`
rdfs11	`C ⊑ D`, `D ⊑ E` (subClassOf)	`C subClassOf E`
rdfs4a/4b	`x p y`	`x a rdfs:Resource`, `y a rdfs:Resource`
rdf1	`x p y`	`p a rdf:Property`
rdfs6 / 8 / 10	`p a rdf:Property` / `C a rdfs:Class`	`p ⊑ p` / `C ⊑ rdfs:Resource` / `C ⊑ C`
se1 / se2	`x p y`	`x p _:b` / `_:b p y` (thay bằng blank)

Format trả lời (như thầy): `:a a :D . <= :a :r :b . :r rdfs:domain :D .` — mỗi triple suy ra ghi **luật + 2 triple cơ sở**, đánh số D1, D2… để dùng lại. **Đúng/sai:** suy ra được (hoặc có sẵn) ⇒ **TRUE** + chỉ ra bước; không suy ra được ⇒ **UNKNOWN** (OWA, RDFS không có phủ định) — giải thích vì sao không luật nào sinh ra. Blank node trong câu hỏi = "tồn tại": chỉ ra phép thay `_:x ↦ :Carl`. **Bẫy:** bộ-3 `subP rdfs:domain C` **không** suy ra được bằng luật RDFS (→ UNKNOWN), nhưng ở mức cá thể vẫn "truyền": `x subP y` ⇒ rdfs7 ⇒ rdfs2 ⇒ `x a C` (slide: "inherited by sub properties"); **không** có inverse (hasChild ≠ hasParent⁻¹); predicate không có trong G ⇒ unknown.

2022-C2 (thầy): `:a a :D <= :a :r :b. :r rdfs:domain :D.` · `:a a :F <= :a a :D. :D rdfs:subClassOf :F.` · `:c a :E <= :c a :C. :C rdfs:subClassOf :E.` · `:b a :C <= :a :r :b. :r rdfs:range :C.` · `:b a :E <= :b a :C. :C rdfs:subClassOf :E.`

P2 Ex2 (gia đình): 1 `Father ⊑ Person` TRUE (rdfs11) · 2 `Man ⊑ Person` TRUE (có sẵn) · 3 `Carl a Person` TRUE (rdfs9) · 4 `Carl a Parent` TRUE (range hasFather=Father, rdfs3 → Father ⊑ Parent, rdfs9) · 5 `Carl hasChild Ann` UNKNOWN (không inverse) · 6 `Carl a Man` TRUE (có sẵn) · 7 `Carl a Father` TRUE (rdfs3) · 8 `Child a rdfs:Resource` TRUE (rdfs4a).

D5 · Thiết kế ontology OWL + DL

Câu tiếng Việt	DL	OWL Turtle
C là một loại D	`C ⊑ D`	`:C rdfs:subClassOf :D`
C chính là / được định nghĩa là D	`C ≡ D`	`owl:equivalentClass`
không có cá thể nào vừa C vừa D	`C ⊓ D ⊑ ⊥`	`:C owl:disjointWith :D`
vừa là C vừa là D	`C ⊓ D`	`[owl:intersectionOf (:C :D)]`
C hoặc D; "gồm có C và D" (tên chung)	`C ⊔ D`	`[owl:unionOf (:C :D)]`
không phải C	`¬C`	`[owl:complementOf :C]`
giá trị chỉ có thể là a, b, c	`{a,b,c}`	`[owl:oneOf (:a :b :c)]`
có ít nhất một R là C	`∃R.C`	`[a owl:Restriction; owl:onProperty :R; owl:someValuesFrom :C]`
mọi R (nếu có) đều là C	`∀R.C`	`… owl:allValuesFrom :C`
có R là a	`∃R.{a}`	`… owl:hasValue :a`

Câu tiếng Việt	DL	OWL Turtle
tối thiểu / tối đa / đúng n	≥n R, ≤n R, =n R	owl:minCardinality / maxCardinality / cardinality "n"^^xsd:nonNegativeInteger
tối thiểu n R là C	≥n R.C	owl:minQualifiedCardinality n ; owl:onClass :C
miền của R là C	∃R.⊤ ⊑ C	:R rdfs:domain :C
khoảng của R là C	⊤ ⊑ ∀R.C	:R rdfs:range :C
R là nghịch đảo của S	R ≡ S⁻	:R owl:inverseOf :S
R bắc cầu / đối xứng / hàm	Trans(R), R ≡ R⁻, ⊤ ⊑ ≤1 R	a owl:TransitiveProperty / SymmetricProperty / FunctionalProperty
R tương đương S; R con của S	R ≡ S; R ⊑ S	owl:equivalentProperty; rdfs:subPropertyOf
C gồm đúng A, B tách rời	C ≡ A ⊔ B, A ⊓ B ⊑ ⊥	owl:disjointUnionOf (:A :B)

Khuôn: ① liệt kê lớp, thuộc tính (object/datatype), cá thể → ② mỗi câu đề → 1 tiên đề DL → ③ dịch sang Turtle → ④ khai a owl:Class / owl:ObjectProperty → ⑤ giải thích lựa chọn (vì sao ∃ hay ∀, ⊑ hay ≡).

2023-C2 (thầy): :PT_ĐườngBộ, :PT_ĐườngThủy, :PT_ĐườngKhông rdfs:subClassOf :PT_GiaoThông · :Đường_Giao_Thông owl:unionOf (:Đường_Bộ :Đường_Thủy :Đường_Không) · :vận_hành rdfs:domain :PT_GiaoThông; rdfs:range :Đường_Giao_Thông · :nhiên_liệu rdfs:domain :PT_GiaoThông; rdfs:range [owl:oneOf (:xăng :dầu :điện)] · :PT_LưỡngCư owl:unionOf ([owl:intersectionOf (:Bộ :Thủy)] [owl:intersectionOf (:Bộ :Không)] [owl:intersectionOf (:Thủy :Không)]). DL: LưỡngCư ≡ (Bộ ⊓ Thủy) ⊔ (Bộ ⊓ Không) ⊔ (Thủy ⊓ Không) · ∃vận_hành.⊤ ⊑ PTGT, ⊤ ⊑ ∀vận_hành.ĐườngGT · ∃nhiên_liệu.⊤ ⊑ PTGT, ⊤ ⊑ ∀nhiên_liệu.{xăng, dầu, điện}.

2022-C4: Dog ⊑ Pet, Cat ⊑ Pet, Dog ⊓ Cat ⊑ ⊥ ⇒ :Dog rdfs:subClassOf :Pet . :Cat rdfs:subClassOf :Pet . :Cat owl:disjointWith :Dog .

6390-C2 (mẫu):

```
:dependsOn a owl:ObjectProperty , owl:TransitiveProperty .   # a. Trans(dependsOn)
:dependsOn owl:equivalentProperty :uses .                    # b. dependsOn ≡ uses
:gpp a :CppCompiler . :clangpp a :CppCompiler . :msvc a :CppCompiler .  # c. (hoặc owl:oneOf)
:CppProgram owl:equivalentClass [ owl:intersectionOf ( :Program
# d. Program ⊓ ∃writtenIn.{CPP} ⊓ ∃compiledBy.CppCompiler
   [ a owl:Restriction ; owl:onProperty :writtenIn ; owl:hasValue :CPP ]
   [ a owl:Restriction ; owl:onProperty :compiledBy ; owl:someValuesFrom :CppCompiler ] ) ] .
:CrossPlatformProgram owl:equivalentClass [ owl:intersectionOf ( :Program   # e. Program ⊓ ≥2 runsOn.OS
   [ a owl:Restriction ; owl:onProperty :runsOn ;
     owl:minQualifiedCardinality "2"^^xsd:nonNegativeInteger ; owl:onClass :OperatingSystem ] ) ] .
```

D6 · Suy diễn OWL / reasoner

Có trong KB	Reasoner suy ra
C ⊑ ∀p.F, x a C, x p y	y a F
C ≡ ∃p.F, x p y, y a F	x a C
C ≡ ∃p.{v} (hasValue), x p v	x a C (và ngược lại x a C ⇒ x p v)
C ≡ A ⊓ B, x a A, x a B	x a C
p owl:inverseOf q, x p y	y q x
p Transitive, x p y, y p z	x p z
p Symmetric, x p y	y p x
p Functional, x p y, x p z	y owl:sameAs z
C ⊓ D ⊑ ⊥, x a C, x a D	**mâu thuẫn** (inconsistent)

OWA: ∀p.F **không** suy ra được cho x chỉ từ dữ liệu (có thể còn p-giá trị chưa biết) ⇒ "x a C" với C ≡ ∀p.F là **không xác định**.

2022-C3 (Molly là mèo, Spot không): RDFS không làm được (range chỉ toàn cục: đổi range thành Cat ⇒ cả Spot cũng là mèo). OWL: :Cat rdfs:subClassOf [a owl:Restriction; owl:onProperty bio:offspring; owl:allValuesFrom bio:Cat] (Cat ⊑ ∀offspring.Cat) ⇒ Milo a Cat, Milo offspring Molly ⇒ Molly a Cat; Fido không phải Cat nên không suy gì về Spot. *(Đáp án thầy:* bio:sprungBy owl:inverseOf bio:offspring . bio:Cat owl:equivalentClass [a owl:Restriction; owl:onProperty bio:sprungBy; owl:allValuesFrom bio:Cat] *— ghi cả hai.)*

6390-C3: a) Từ C ≡ ∀p.F và C ⊑ D, E ⇒ [a owl:Restriction; owl:onProperty :p; owl:allValuesFrom :F] rdfs:subClassOf :D , :E (mọi cá thể có mọi p-giá trị là F thì là D và E). Không suy được :y a :C (OWA). b) DL: C ⊑ D ⊓ E, C ≡ ∀p.F; ngữ nghĩa: $C^I \subseteq D^I \cap E^I$, $C^I = \{ x \mid \forall y: (x,y) \in p^I \to y \in F^I \}$.

D12 · Tableau

NNF: C ⊑ D → ¬C ⊔ D · C ≡ D → ¬C ⊔ D và ¬D ⊔ C · ¬(C⊓D) → ¬C ⊔ ¬D · ¬(C⊔D) → ¬C ⊓ ¬D · ¬∃R.C → ∀R.¬C · ¬∀R.C → ∃R.¬C · C ⊓ D ⊑ ⊥ → ¬C ⊔ ¬D · domain ∃R.⊤ ⊑ C → ∀R.⊥ ⊔ C · range → ∀R.C.

Nr	Có	Làm
1	C(a)	thêm C(a)
2	R(a,b)	thêm R(a,b)
3	C (tiên đề TBox)	chọn cá thể a, thêm C(a)
4	(C ⊓ D)(a)	thêm C(a), D(a)
5	(C ⊔ D)(a)	**tách nhánh**: T1 + C(a), T2 + D(a)
6	(∃R.C)(a)	thêm R(a,b), C(b) với b **mới**
7	(∀R.C)(a)	với mọi b có R(a,b): thêm C(b)

Quy trình: ① đưa câu hỏi về **consistency**: kiểm KB ⊨ C(a) ⇒ thêm ¬C(a); kiểm C ⊑ D ⇒ thêm (C ⊓ ¬D)(x) với x mới → ② chuyển mọi tiên đề sang NNF → ③ áp luật (ưu tiên 1,2,4,6,7 trước 5) → ④ nhánh **đóng** khi có X(a) và ¬X(a) → ⑤ **mọi nhánh đóng** ⇒ KB+phủ định mâu thuẫn ⇒ điều cần chứng minh **đúng**; còn nhánh mở không áp được luật ⇒ **không** suy ra. **Blocking:** luật 6 áp cho cá thể a một lần thì chặn (tránh vô hạn).

Mẫu: KB A ⊑ B, A(a). Hỏi B(a)? Thêm ¬B(a); NNF ¬A ⊔ B; luật 3 tại a: (¬A ⊔ B)(a); luật 5: T1 ¬A(a) ✗ (với A(a)), T2 B(a) ✗ (với ¬B(a)) ⇒ mọi nhánh đóng ⇒ B(a) đúng.

D7 · Datalog

Khuôn: B0 = dữ kiện cho sẵn · B(i+1) = Bi + mọi dữ kiện **mới** suy ra từ Bi bằng 1 lần áp các luật · dừng khi một bước không thêm gì ⇒ **điểm cố định** ở bước cuối có thêm. Viết rõ "B1: + {…}". Cây chứng minh: gốc = khẳng định; con = các tiền đề của luật đã dùng; lá = dữ kiện B0.

2023-C3 (thầy): B0: {1C(A,B), 1C(B,C), 2C(D,C)} B1: + {2C(C,D), Đi(A,B), Đi(B,C), Đi(D,C)} B2: + {Đi(C,D), Đi(A,C)} B3: + {Đi(B,D), Đi(A,D), Đi(C,C), Đi(D,D)} — B4 không thêm ⇒ **điểm cố định ở B3**. *(1C = Đường-một-chiều, 2C = Đường-hai-chiều, Đi = Có-đường-đi)*

```
Có-đường-đi(A,D)                                  [luật bắc cầu]
├── Có-đường-đi(A,C)                               [luật bắc cầu]
│   ├── Có-đường-đi(A,B) ← Đường-một-chiều(A,B)
│   └── Có-đường-đi(B,C) ← Đường-một-chiều(B,C)
└── Có-đường-đi(C,D) ← Đường-hai-chiều(C,D) ← Đường-hai-chiều(D,C)   [luật đối xứng]
```

D8 · SPARQL (câu 2.5 điểm)

```
PREFIX : <http://example.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
SELECT DISTINCT ?x ?name (COUNT(?y) AS ?n)
WHERE {
  ?x a :C ; :p ?y .                       # khớp mẫu (cùng subject dùng ;)
  OPTIONAL { ?x rdfs:label ?name }        # "nếu có, để trống nếu không"
  FILTER (?y > 1990 && ?x != :A)
  FILTER NOT EXISTS { ?x :bad ?z }        # "không có"
}
GROUP BY ?x ?name  HAVING (COUNT(?y) >= 2)
ORDER BY DESC(?n)  LIMIT 10
```

Đề nói	Dùng
"nếu có / để trống nếu không"	`OPTIONAL { }`
"A hoặc B"	`{ … } UNION { … }` hoặc `FILTER (?t = :A || ?t = :B)` hoặc `VALUES ?t { :A :B }`
"không có / chưa có"	`FILTER NOT EXISTS { }` / `MINUS { }` / `OPTIONAL + FILTER(!bound(?v))`
"đếm / mỗi / tổng / trung bình"	`GROUP BY` + `COUNT / SUM / AVG / MIN / MAX`; biến SELECT không gộp phải có trong GROUP BY
"ít nhất 2 / nhiều hơn"	`HAVING (COUNT(?x) >= 2)`
"kể cả gián tiếp / bắc cầu"	path `:p+` (≥1), `:p*` (≥0), `rdfs:subClassOf*`, `^:p` (ngược), `:p/:q` (nối)
"khẳng định / tạo triple mới"	`CONSTRUCT { ?x a :New } WHERE { … }`
"có … không?"	`ASK { … }`
"chứa / bắt đầu bằng"	`FILTER regex(?s, "abc", "i")`, `CONTAINS(?s,"x")`, `STRSTARTS`
"tiếng Anh"	`FILTER (lang(?t) = "en")`
"ghép tên thành 1 cột"	`GROUP_CONCAT(?n; separator=", ")`
"sắp xếp / top k"	`ORDER BY DESC(?v) LIMIT k`

Đề cũ — đáp án:

```
# 2023-C4a: thủy phi cơ + trực thăng, kèm nhãn nếu có
SELECT * WHERE { { ?p a ?t } OPTIONAL { ?p rdfs:label ?label }
                 FILTER (?t = :Seaplane || ?t = :Helicopter) }
# → :Caspar :Seaplane | :Convair :Seaplane | :Mi26 :Helicopter
"Mil Mi-26"

# 2022-C5a: mèo của Anna và con của chúng
SELECT ?pet WHERE { { bio:Anna bio:own ?pet . ?pet a bio:Cat .
}
  UNION { bio:Anna bio:own ?x . ?x a bio:Cat . ?x bio:offspring
?pet . } }
# → bio:Milo, bio:Kitty

# 6390-C4a: NNLT có con trỏ hoặc lớp       → :CPP, :C, :Java
SELECT DISTINCT ?pl WHERE { ?pl a :PL ; :hasFeature ?f . FILTER
(?f = :pointer || ?f = :class) }
# 6390-C4b: NNLT + người sáng lập nếu có    → Lisp, Ruby: ?
creator trống
SELECT ?pl ?creator WHERE { ?pl a :PL . OPTIONAL { ?pl :creator
?creator } }
# 6390-C4c: khẳng định OOPL                 → CPP, Java, Python a
:OOPL
CONSTRUCT { ?pl a :OOPL } WHERE { ?pl :hasFeature :class }
```

Bẫy: quên `.` giữa các triple · quên PREFIX · FILTER so sánh IRI dùng `=`, chuỗi dùng `"…"` · biến trong OPTIONAL có thể trống · đếm cả bài chưa có: `COUNT(?r)` + OPTIONAL (không phải `COUNT(*)`) · literal số không có ngoặc kép.

D9 · Đại số SPARQL

Cú pháp	Đại số
nhóm triple `{ t1 . t2 }`	`BGP(t1 . t2)`
hai nhóm cạnh nhau	`Join(P1, P2)`
`P OPTIONAL { Q }`	`LeftJoin(P, Q)` (FILTER trong OPTIONAL → `LeftJoin(P, Q, expr)`)
`{P} UNION {Q}`	`Union(P, Q)`
`FILTER(e)`	`Filter(e, …)` bọc **cả nhóm** chứa nó (đặt ngoài cùng nhóm)
`GROUP BY` + hàm gộp	`Group(…)`, `Aggregate(…)`
`SELECT ?x ?y`	`Project(…, {?x, ?y})` · `DISTINCT` → `Distinct` · `ORDER BY` → `OrderBy` · `LIMIT` → `Slice`

Định nghĩa: μ = ánh xạ biến → giá trị. Hai μ **tương thích** nếu biến chung cùng giá trị. `Join` = {μ1 ∪ μ2 tương thích} · `LeftJoin` = Join ∪ {μ1 không có μ2 nào tương thích} · `Union` = Ω1 ∪ Ω2 · `Filter` = giữ μ làm e đúng · `Project` = chỉ giữ các biến chọn.

Khuôn: ① viết biểu thức (từ trong ra ngoài) → ② tính **BGP** trước, mỗi bước 1 bảng → ③ áp lần lượt Join/LeftJoin/Union → Filter → Project → ④ bảng kết quả cuối.

2023-C4b (thầy): `Filter(?t = :Seaplane || ?t = :Helicopter, LeftJoin(BGP(?p a ?t), BGP(?p rdfs:label ?label)))`

Bước	Kết quả
BGP(?p a ?t)	(Caspar,Seaplane) (Convair,Seaplane) (Su27,JetFighter) (Mig35,JetFighter) (Mi26,Helicopter)
BGP(?p rdfs:label ?label)	(Mi26,"Mil Mi-26")
LeftJoin	5 dòng trên, Mi26 có thêm ?label = "Mil Mi-26"
Filter	Caspar Seaplane – · Convair Seaplane – · Mi26 Helicopter "Mil Mi-26"

2022-C5b (thầy): `Project(Union(BGP(Anna own ?pet . ?pet a Cat), BGP(Anna own ?x . ?x a Cat . ?x offspring ?pet)), {?pet})` · B1: {?pet→Milo} · B2: {?x→Milo, ?pet→Kitty} · Union: 2 dòng (dòng 1 ?x trống) · Project ?pet: **Milo, Kitty**.

6390-C4d: `Distinct(Project(Filter(?f = :pointer || ?f = :class, BGP(?pl a :PL . ?pl :hasFeature ?f)), {?pl}))` · BGP: 11 dòng (CPP×5, C×3, Java×2, Lisp×1 — Python không a `:PL`, Ruby không có feature) · Filter: (CPP,pointer) (CPP,class) (C,pointer) (Java,class) · Project + Distinct: **CPP, C, Java**.

D10 · SHACL

```
:XShape a sh:NodeShape ;
  sh:targetClass :X ;                  # nút được kiểm = mọi ?n
a :X
  sh:property [ sh:path :p ;
                sh:minCount 1 ; sh:maxCount 1 ;          # bắt
buộc / tối đa
                sh:datatype xsd:integer ;                # kiểu
literal
                sh:minExclusive 1900 ] ;                 # >
1900 (minInclusive: ≥)
  sh:property [ sh:path :q ; sh:class :Y ; sh:in ( :a :b ) ;
sh:pattern "^A" ] .
```

Ràng buộc	Nghĩa
`sh:targetClass` / `targetNode` / `targetSubjectsOf` / `targetObjectsOf`	chọn nút được kiểm
`sh:minCount` / `sh:maxCount`	số giá trị tối thiểu / tối đa
`sh:datatype` / `sh:class` / `sh:nodeKind`	kiểu literal / lớp của giá trị / IRI-Literal-BlankNode
`sh:minInclusive` `minExclusive` `maxInclusive` `maxExclusive`	≥ > ≤ <
`sh:in` / `sh:hasValue` / `sh:pattern` / `sh:minLength`	thuộc danh sách / phải có giá trị / regex / độ dài

Trả lời "nút nào được kiểm / vi phạm": ① liệt kê nút khớp target (chỉ những nút có `a :X` — **không** suy diễn ngoài subClassOf) → ② kiểm từng ràng buộc cho từng nút → ③ nêu nút vi phạm + lý do → ④ đề xuất thêm dữ liệu. SHACL = **thế giới đóng** (thiếu là vi phạm), khác OWL (thiếu là chưa biết).

2023-C5 (thầy): `:SeaPlaneShape a sh:NodeShape; sh:targetClass :Seaplane; sh:property [sh:path :flight; sh:minExclusive 1900].` → Caspar (flight 1900, không > 1900) **vi phạm**. `:JetFighterShape … sh:targetClass :JetFighter; sh:property [sh:path :crew; sh:minCount 1]; sh:property [sh:path :speed; sh:minCount 1].` → Su27 thiếu speed, Mig35 thiếu crew. **6390-C5:** `:PLFeatures a sh:NodeShape; sh:targetClass :PL; sh:property [sh:path :hasFeature; sh:minCount 1].` Kiểm: CPP, C, Java, Lisp, Ruby; **Python không được kiểm** (không a `:PL`). **Ruby vi phạm** (0 tính năng) → thêm `:Ruby :hasFeature :class .`

D11 · Lý thuyết mở

Chức năng của reasoner trong KB (viết theo ý, mỗi ý 1 ví dụ):

1. **Kiểm tra tính nhất quán** (consistency): KB có mâu thuẫn không — vd `Cat ⊓ Dog ⊑ ⊥` mà `Tom a Cat, Dog`.
2. **Kiểm tra lớp thỏa được** (satisfiability): lớp có thể có cá thể không — lớp con của hai lớp tách rời là rỗng.

3. **Phân loại** (classification): tính cây subsumption đầy đủ — suy ra `Father ⊑ Person`.
4. **Realization / kiểm tra cá thể**: tìm mọi lớp của cá thể — `Carl a Father` ⇒ `Carl a Person`.
5. **Suy ra tri thức ẩn** (entailment, materialization): sinh triple ngầm định — rdfs2/3/7/9, inverseOf, transitive.
6. **Hỗ trợ trả lời truy vấn**: truy vấn trả về cả kết quả suy ra (SPARQL entailment regime).
7. **Giải thích** (justification): chỉ ra các tiên đề dẫn tới kết luận / mâu thuẫn — sửa lỗi ontology. Cơ chế: RDFS/OWL RL dùng luật (forward chaining), OWL DL dùng **tableau**. Reasoner: HermiT, Pellet, FaCT++. Theo **OWA**: chưa suy ra được ≠ sai.

Ngắn gọn khác: OWA (thiếu = chưa biết; OWL/RDFS) ↔ **CWA** (thiếu = sai; CSDL, SHACL) · **không UNA**: hai IRI có thể cùng một vật (owl:sameAs) · **LOD 4 nguyên tắc**: dùng URI · HTTP URI · trả về thông tin hữu ích (RDF, SPARQL) · liên kết tới URI khác · **5 sao**: web (giấy phép mở) → máy đọc được → định dạng mở → RDF/URI → liên kết dữ liệu khác · **RDFS vs OWL**: OWL thêm phủ định, disjoint, cardinality, inverse/transitive, định nghĩa lớp bằng điều kiện · **SHACL vs OWL**: kiểm dữ liệu (đóng) ↔ suy diễn (mở).